DATABRICKS DATA ENGINEER
ASSOCIATE CERTIFICATION
Chapter 5 — Advanced Spark Declarative Pipelines

Complete Study & Exam Preparation Guide

Covers: Multi-Flows • Liquid Clustering • Data Quality Expectations • Multiplex • CDC • Delta UniForm

CHAPTER 5 OVERVIEW

Chapter 5 covers the advanced concepts of Spark Declarative Pipelines (SDP) — the powerful Databricks feature built on top of Delta Live Tables. This chapter is divided into four major lectures, each covering a critical real-world pattern you must understand for the certification exam.

#	Lecture Topic	Core Concepts
1	Multi-Flows, Expectations & Liquid Clustering	Default/Explicit Flows, Multi-Flow pattern, CLUSTER BY, Expectations
2	Multiplex, Delta Sinks & Iceberg Reads	Fan-out pattern, Delta Sink, Delta UniForm
3	Change Data Capture (CDC) Review	SCD Type 1, SCD Type 2, AUTO CDC INTO
4	Advanced Data Quality Checks	Cross-table validation, Quarantine pattern, Schema evolution

LECTURE 1 — Multi-Flows, Expectations & Liquid Clustering

Key Terms — Lecture 1

Flow	The basic unit of work in a Spark Declarative Pipeline. A Flow = Query + Target. It reads data, transforms it, and writes to a Streaming Table or Materialized View.
Default Flow	Automatically created when you write CREATE OR REFRESH STREAMING TABLE ... AS SELECT. The flow takes the same name as the table.
Explicit Flow	A flow defined separately using CREATE FLOW. Required when multiple sources need to write to the same target table.
Checkpoint	A bookmark stored per flow that tracks exactly how far the flow has read from its source. Enables reliable restarts without reprocessing old data.
Incremental Mode	Processes only NEW records since the last pipeline run, using the checkpoint. Efficient for large datasets.
Full Refresh Mode	Discards the checkpoint and reprocesses ALL records from scratch. Used when transformation logic changes.
Multi-Flow Pattern	Using multiple Explicit Flows that all write to the same target Streaming Table — one flow per source.
Liquid Clustering	A Delta Lake optimization that replaces Hive partitioning and Z-Ordering. Automatically reorganizes data files for fast query performance.
CLUSTER BY AUTO	Databricks automatically chooses the best clustering keys based on observed query patterns.
CONSTRAINT ... EXPECT	The syntax to define a data quality rule on a Streaming Table inside SDP.
ON VIOLATION WARN	Invalid rows are KEPT in the table but a warning metric is logged. Default if no clause specified.

ON VIOLATION DROP ROW	Invalid rows are REMOVED from the table. Dropped count is tracked in metrics.

ON VIOLATION FAIL UPDATE	The ENTIRE pipeline update is HALTED when even one row violates the rule. For critical fields.

A. Understanding Flows in Spark Declarative Pipelines

A1. What is a Flow?

A **Flow** is the smallest unit of execution in SDP. Think of it like a **worker assigned to one specific task** — it reads from a source, transforms the data (filter, join, aggregate), and writes the result to a target.

Simple formula to remember:

Formula	FLOW = QUERY + TARGET Query = What to read and how to transform it (SQL / DataFrame logic) Target = Where to write the result (Streaming Table or Materialized View)

A2. Processing Modes — Incremental vs Full Refresh

Mode	How It Works	When to Use
Incremental Mode	Reads only NEW records since the last run using a checkpoint. Highly efficient.	Streaming sources: Kafka, Auto Loader, append-only tables
Full Refresh Mode	Discards checkpoint, reads ALL records from scratch. Overwrites the target table.	After changing transformation logic or fixing data quality issues

A3. Flow Checkpoint — 4 Rules to Remember

Checkpoints are critical for the exam. Memorize these four rules:

1 Flow Name = Checkpoint Identity

The checkpoint is stored under the flow's name. Change the name → new checkpoint.

2 Renaming Resets Progress

Renaming a flow creates a brand-new checkpoint. The flow will reprocess all data from the beginning.

3 Independent Progression

Each flow advances at its own pace. A slow flow does NOT delay other flows in the same pipeline.

4 Failure Isolation

If one flow fails, all other flows continue processing normally. Failures are scoped to individual flows.

A4. Default Flow vs Explicit Flow

Type	How Created	SQL Syntax	When to Use
Default Flow	Auto-created with the table. Takes the table's name.	CREATE OR REFRESH STREAMING TABLE target AS SELECT * FROM STREAM source;	Simple pipelines — one source, one target.
Explicit Flow	Defined separately with CREATE FLOW.	CREATE OR REFRESH STREAMING TABLE target; CREATE FLOW my_flow AS INSERT INTO target BY NAME SELECT * FROM STREAM source;	When multiple flows write to the same target.

B. The Multi-Flow Pattern

B1. Writing Multiple Sources into One Target

The **multi-flow pattern** lets you define multiple Explicit Flows that all target the SAME Streaming Table. Each flow independently reads from a different source (CSV, JSON, Kafka, etc.) and appends its records to the shared table.

Real-World Analogy

Imagine a single inbox (the target table) that receives mail from three different postal routes (three flows). Each route has its own tracker (checkpoint). If one route is delayed, the other two keep delivering independently.

B2. Multi-Flow vs UNION — Why It Matters

This is a high-probability exam question. Understand WHY multi-flow is better:

UNION Approach — Problems

- All sources share ONE checkpoint
- Adding a new source = FULL REFRESH (reprocess everything!)
- One source failure blocks ALL others
- Poor lineage tracking per source
- Hard to scale as source count grows

Multi-Flow Approach — Benefits

- Each flow has its OWN independent checkpoint
- New sources added WITHOUT full refresh
- One flow failure NEVER affects other flows
- Clear per-source lineage and monitoring
- Better scalability and maintainability

C. Data Quality Expectations

Data Quality Expectations are **inline rules** defined on a Streaming Table. When a record violates a rule, you choose one of three **violation modes** to determine what happens. All flows writing to a table automatically follow the same expectations.

C1. Three Violation Modes

Mode	What Happens	When to Use	Example Syntax
WARN (default)	Invalid rows are KEPT. Warning metric logged in pipeline event log.	Monitoring unusual data without blocking ingestion.	CONSTRAINT valid_email EXPECT (email IS NOT NULL)
DROP ROW	Invalid rows are REMOVED. Dropped count tracked in metrics.	Clean datasets where invalid rows are expected and can be safely discarded.	CONSTRAINT valid_qty EXPECT (qty > 0) ON VIOLATION DROP ROW
FAIL UPDATE	ENTIRE pipeline update is halted at first violation.	Critical fields (e.g., primary key) where ANY violation indicates a serious problem.	CONSTRAINT not_null_id EXPECT (id IS NOT NULL) ON VIOLATION FAIL UPDATE

⚠ **EXAM TIP** Data Quality Expectations CANNOT be defined on a Flow. When using the multi-flow pattern, constraints must be defined on the TARGET TABLE — not inside the CREATE FLOW definition. The table enforces quality centrally for ALL flows writing to it.

C2. Recommended Strategy — Layered Approach

- **FAIL UPDATE** → For critical business keys (e.g., order_id, customer_id must never be null)
- **DROP ROW** → For quality issues that can be filtered out (e.g., negative quantities)
- **WARN** → For monitoring unusual but potentially valid data patterns (e.g., quantities > 1000)

D. Liquid Clustering in Spark Declarative Pipelines

D1. Evolution of Data Layout Optimization

Approach	How It Works	Problem
Hive Partitioning	Creates rigid directory structure by partition column (e.g., /year=2024/month=01/).	Rigid — full rewrite needed to change partition keys.
Z-Ordering	Sorts data within partitions to co-locate related values.	Manual OPTIMIZE required after each run. Full rewrite every time.
Liquid Clustering ☑	Incrementally reorganizes ONLY new/unclustered data. Keys changeable anytime.	None of the above! This is the recommended approach.

🏛 **Analogy** Think of Liquid Clustering as a library that continuously rearranges its shelves based on what readers actually borrow. Hive partitioning locks books into fixed rooms by genre. Z-Ordering manually sorts within rooms. Liquid Clustering observes patterns and reorganizes automatically over time.

D2. Three Key Properties of Liquid Clustering

Property	What It Means
Incremental	Only optimizes NEW or unclustered data files — never rewrites already-clustered files. Efficient for streaming and write-heavy workloads.
Flexible	Clustering keys can be changed at ANY TIME without a full table rewrite. Adapts to evolving query patterns.
Self-Tuning	With CLUSTER BY AUTO, Databricks automatically selects optimal keys based on observed query usage.

D3. CLUSTER BY AUTO vs CLUSTER BY (columns)

CLUSTER BY AUTO

- Databricks learns and decides the best keys
- Keys evolve as query patterns change
- Zero manual tuning needed
- **Best for: New or evolving tables**

⚠ *Starts UNSET — Databricks needs to observe queries first*

CLUSTER BY (columns)

- You explicitly specify clustering keys
- Full control — predictable layout
- No learning period needed
- **Best for: Stable, well-known query patterns**

☑ *Recommended for regulatory/performance-critical tables*

LECTURE 2 — Multiplex, Delta Sinks & Iceberg Reads

Key Terms — Lecture 2

Multiplex Pattern	A design pattern where ALL event types from a single transport (e.g., one Kafka topic) are ingested into ONE bronze table, then filtered downstream by event type.
Fan-Out	The process of splitting one ingested bronze table into multiple domain-specific tables by filtering on an event type field.
Sink	A mechanism in SDP allowing pipeline output to be written to targets OUTSIDE the pipeline's managed scope (external Delta tables, Kafka, Event Hubs, or custom targets).
Delta Sink	A sink writing to an external Delta table not managed by the pipeline. Required for enabling Iceberg compatibility via UniForm.
append_flow	The ONLY flow type that can write to a Sink. Defined using @dp.append_flow Python decorator. No SQL equivalent exists for sinks.
Delta UniForm	Universal Format — auto-generates Apache Iceberg metadata on the same Parquet files, enabling cross-platform reads without data duplication.
Deletion Vectors	A Delta performance feature using soft-delete markers. Must be DISABLED (delta.enableDeletionVectors = false) for Iceberg/UniForm compatibility.
Column Mapping	A Delta table property (delta.columnMapping.mode = 'name') required for Iceberg compatibility. Permanent — cannot be dropped once enabled.

A. The Multiplex Pattern

Production environments often receive **multiple event types through a single data transport** (one Kafka topic, one S3 path, one message queue). Without a strategy, this means N pipelines for N event types — N separate checkpoints and N separate source scans.

The Multiplex pattern solves this efficiently:

✗ Without Multiplex	☑ With Multiplex

N event types → N separate pipelines N separate checkpoints and source scans Source change must be applied N times	N event types → 1 ingestion pipeline (Bronze Table) 1 checkpoint, 1 source scan shared across all domains Source change applied in ONE place

Multiplex Flow

Mixed Event Stream → Bronze Table (all types, VARIANT PAYLOAD) → WHERE type = A → Domain A Table
WHERE type = B → Domain B Table
→ WHERE type = C → Domain C Table

B. Sinks — Writing to External Targets

By default, all pipeline outputs (Streaming Tables, Materialized Views) are **managed and owned by the pipeline**. A **Sink** breaks this boundary intentionally — it lets you write streaming data to an **external target outside the pipeline's lifecycle**.

⚠ CRITICAL EXAM POINT

ONLY the Python API supports Sinks. SQL is NOT supported. ONLY append_flow can write to a Sink — no other flow type can.

B1. Four Types of Sinks

Sink Type	Destination	Use Case
Delta Table Sink	Unity Catalog managed or external Delta tables (by path or table name)	Writing pipeline output for cross-platform access, enabling Iceberg UniForm
Apache Kafka Sink	Kafka topics	Low-latency operational use cases, Reverse ETL out of Databricks
Azure Event Hubs Sink	Azure Event Hubs (uses Kafka interface format)	Real-time event streaming, fraud detection, recommendations
Python Custom Sink	ANY data store via PySpark custom data sources	Maximum flexibility for any non-standard destination

B2. Managed Tables vs Sinks

Feature	Managed Table (Default)	Sink
Data location	Within Unity Catalog (pipeline scope)	External systems outside Databricks or pipeline scope
Lineage tracking	Full pipeline lineage	Not tracked by pipeline
Expectations / CDC	Supported	NOT supported — append only

Iceberg / UniForm	NOT possible on pipeline-managed tables	Possible — plain Delta tables can have UniForm enabled
Python API required?	No (SQL or Python)	YES — only Python (@dp.append_flow)

B3. Delta Sink — Implementation (2 Steps)

Step 1 — Register the Sink:

```
from pyspark import pipelines as dp

dp.create_sink(
    name    = "my_sink",
    format  = "delta",
    options = { "tableName": "catalog.schema.table" }
)
```

Step 2 — Write to the Sink using append_flow:

```
@dp.append_flow(name = "my_sink_flow", target = "my_sink")
def my_sink_flow():
    return spark.readStream.table("schema.source_table")

# Checkpointing handled automatically by append_flow
# Only NEW records written per run — no overwrites
```

C. Iceberg Reads via Delta UniForm

Delta UniForm solves the cross-platform interoperability problem: different tools (Snowflake, Trino, Athena, OSS Spark) understand Iceberg — but NOT Delta natively. UniForm generates Iceberg metadata asynchronously on the same Parquet files, so external tools can read your Delta table as if it were a native Iceberg table.

Key Insight Delta + Iceberg both run on Parquet files. UniForm simply adds an Iceberg metadata layer on top of the SAME files. Result: ONE dataset → TWO logical views → ZERO data duplication.

C1. Four Table Properties Required for UniForm

#	Property	Value	Why Required
1	delta.enableDeletionVectors	'false'	Iceberg v2 cannot represent Delta's soft-delete markers. Must disable.
2	delta.columnMapping.mode	'name'	Ensures column IDs are consistent between Delta and Iceberg schemas. PERMANENT — cannot be dropped.
3	delta.enableIcebergCompatV2	'true'	Activates write protocol compatible with Iceberg v2.

4	delta.universalFormat.enabledFormats	'iceberg'	Triggers async Iceberg metadata generation after every Delta commit.

⚠ **CRITICAL EXAM POINT** Pipeline-managed Streaming Tables and Materialized Views CANNOT have Iceberg reads enabled. ONLY plain external Delta tables (created via a Delta Sink) support UniForm. This is WHY Delta Sinks exist — they are the bridge for cross-platform Iceberg access.

C2. UniForm Limitations — Know These for the Exam

- Iceberg clients can only READ via Iceberg. All WRITES must go through Delta.
- Deletion Vectors must be disabled — incompatible with UniForm.
- Streaming Tables and Materialized Views CANNOT use UniForm directly — use Delta Sink.
- Column mapping is PERMANENT — cannot be removed once enabled.
- Requires Databricks Runtime 14.3 LTS or above for any writing client.
- Only triggered when table is accessed by NAME (not path).

C3. How the Three Concepts Connect (Full Pattern)

Step	Concept	Problem Solved	Output
1	Multiplex Pattern	Multiple event types in one source	Domain-specific Silver tables
2	Delta Sink	Need external table for cross-platform access	Plain Delta table outside pipeline scope
3	Delta UniForm	External tools need Iceberg format access	Dual-protocol table accessible by Snowflake, Trino, Athena

LECTURE 3 — Change Data Capture (CDC) Review

Key Terms — Lecture 3

Change Data Capture (CDC)	A technique to track and capture changes (INSERT, UPDATE, DELETE) in a source system and apply them to a target table to keep it synchronized.
Slowly Changing Dimension (SCD)	A concept describing how changes in data (like a customer address update) are handled in the target table over time.
SCD Type 1	Overwrites the existing row with new values. NO history is kept — only the current state is stored.
SCD Type 2	Preserves history by creating a NEW row for each change while marking the old row as inactive. Full audit trail maintained.
__START_AT	SCD Type 2 metadata column — timestamp when this row became active/valid.
__END_AT	SCD Type 2 metadata column — timestamp when this row became inactive. NULL means currently active.
AUTO CDC INTO	Databricks declarative syntax for implementing CDC. Replaced the older APPLY CHANGES INTO syntax (but syntax/functionality are identical).
KEYS	Column(s) that uniquely identify a record. Used by AUTO CDC INTO to match source rows to target rows. Can be a single column or composite key.
SEQUENCE BY	Column used to order CDC events chronologically. Ensures events are applied in the correct order even if they arrive out of order.
Composite Key	Using MULTIPLE columns together to uniquely identify a record — e.g., KEYS (CustomerID, OrderID).

A. What is Change Data Capture (CDC)?

CDC is used to track changes in source systems (databases, lakehouses, data warehouses) and **apply those changes to a target table** so it always reflects the latest — or historical — state of the source.

A classic example: a **customers table** where addresses change, accounts are deleted, and new customers sign up. CDC automates the synchronization so the target table never goes stale.

B. SCD Type 1 — Current State Only

SCD Type 1 is the simpler approach. When data changes, the existing row is simply OVERWRITTEN. No history is retained.

Action	What Happens in Target Table
INSERT	New row added to the target table with the new customer's data.
UPDATE	Existing row is overwritten with the new values. OLD values are permanently lost.
DELETE	Row is removed from the target table entirely. No trace remains.

Use SCD Type 1 when... • Only the most current, up-to-date data is needed for operations • Storage efficiency is important • Historical tracking is NOT required • Regulatory compliance does NOT require audit trails

C. SCD Type 2 — Full Historical Tracking

SCD Type 2 preserves a complete audit trail. Instead of overwriting, it marks the old record as 'closed' and inserts a new 'active' record. Deleted records become 'inactive' with an end timestamp.

C1. Two Special Metadata Columns

Column	Meaning	Value for Active Records	Value for Historical Records
__START_AT	When this version of the row became active	Timestamp of when this version was created	Timestamp of when this version was created
__END_AT	When this version became inactive (superseded or deleted)	NULL ← means CURRENTLY ACTIVE	Non-null timestamp ← means CLOSED/HISTORICAL

Action	What Happens in Target Table
INSERT	New row added. __START_AT = insert timestamp. __END_AT = NULL (active).
UPDATE	Old row: __END_AT is populated (closed). New row: inserted with updated values, __END_AT = NULL.

DELETE	Soft delete — row remains but __END_AT is populated. No record is physically removed.

Use SCD Type 2 when... • Historical analysis of how data changed over time is required • Compliance / audit trails are mandatory • Point-in-time reporting is needed (e.g., what was customer's address on 01/Jan/2024?) • Advanced analytics on changing dimensions

D. AUTO CDC INTO — Declarative CDC in SDP

AUTO CDC INTO is Databricks' declarative way to implement CDC inside Spark Declarative Pipelines. It eliminates complex MERGE INTO logic and handles event sequencing automatically.

Syntax Note AUTO CDC INTO was previously called APPLY CHANGES INTO. The syntax and functionality are IDENTICAL — just a name change.

AUTO CDC INTO — Full Syntax Breakdown (SCD Type 1)

```
CREATE OR REFRESH STREAMING TABLE customers;

CREATE FLOW scd_type_1_flow AS
AUTO CDC INTO customers            -- Target table
  FROM STREAM updates              -- Source stream with CDC events
  KEYS (CustomerID)                -- Unique key(s) to match rows
  APPLY AS DELETE WHEN             -- How to detect DELETE events
    operation = 'DELETE'
  SEQUENCE BY ProcessDate          -- Column to order events chronologically
  COLUMNS * EXCEPT (operation)    -- Include all columns except metadata
  STORED AS SCD TYPE 1;           -- SCD pattern: Type 1 (default) or Type 2
```

AUTO CDC INTO — Clause-by-Clause Explanation

Clause	What It Does
AUTO CDC INTO customers	Specifies the target streaming table where changes will be applied.
FROM STREAM updates	The source stream containing incoming CDC events (INSERT, UPDATE, DELETE rows).
KEYS (CustomerID)	Identifies which column(s) uniquely identify a record. Used to match source rows to target rows.
APPLY AS DELETE WHEN operation = 'DELETE'	Tells SDP which rows represent a deletion (often marked by a special column value).
SEQUENCE BY ProcessDate	Ensures events are applied in chronological order. Critical for out-of-order event handling.

COLUMNS * EXCEPT (operation)	Include all data columns but exclude operational metadata (like the operation type column itself).
STORED AS SCD TYPE 1	Specifies the SCD pattern. Change to SCD TYPE 2 for historical tracking.

Composite Key A composite key uses MULTIPLE columns together to uniquely identify a record. Example: KEYS (CustomerID, OrderID) — useful when one column alone is not unique.

D1. SCD Type 1 vs SCD Type 2 — Decision Framework

Decision Factor	Choose SCD Type 1	Choose SCD Type 2
History needed?	No — only current state	Yes — full change history required
Storage concern?	Efficient — one row per entity	Higher storage — multiple rows per entity
Compliance/audit?	Not required	Required — regulatory or business need
Point-in-time queries?	Not needed	Needed (e.g., 'what was state on Jan 1?')
STORED AS clause	STORED AS SCD TYPE 1 (or omit — default)	STORED AS SCD TYPE 2

LECTURE 4 — Advanced Data Quality Checks

Key Terms — Lecture 4

NULL-Tolerant Constraint	A constraint written with CASE WHEN ... IS NOT NULL THEN ... ELSE TRUE END to avoid false violations when a column may be absent or was added after pipeline start.
Cross-Table Validation	Expectations defined on Materialized Views that validate data ACROSS multiple tables (row count checks, missing record detection, primary key uniqueness).
Row Count Validation	A cross-table check comparing COUNT(*) between two tables to ensure no records were silently dropped.
Missing Record Detection	Using a LEFT OUTER JOIN to find records present in a validation copy but absent in the target table.
Primary Key Uniqueness	Grouping by the PK and checking COUNT(*) = 1 to ensure no duplicate rows exist.
schemaHints	An option in read_files() to pre-declare columns that will appear in FUTURE files. Old records get NULL for those columns.
_rescued_data	A special column that captures any fields arriving outside the declared schema as JSON. Acts as a catch-all — nothing is silently discarded.
Bronze Layer	The raw ingestion layer in the Medallion architecture. Best practice: store all fields as STRING to avoid type-mismatch failures.
TRY_CAST	A safe casting function that returns NULL instead of raising an error when a value cannot be converted to the target type.
Quarantine Pattern	A pipeline design that routes ALL records (valid AND invalid) to a single table with an is_quarantined flag, then splits them downstream. Achieves ZERO data loss.
Inverse Logic	The is_quarantined column formula: NOT(all rules pass). If ANY rule fails → is_quarantined = TRUE.

Partition Pruning	Query optimization where only relevant partitions are scanned. The quarantine table is partitioned by is_quarantined so clean/invalid queries only scan their partition.

A. Why Advanced Quality Expectations?

A1. The Limits of Basic NOT NULL Checks

A basic **NOT NULL** check only confirms that a field is **present** — it does NOT confirm the value is **correct**. Real production data has many more quality problems:

Problem Type	Example	NOT NULL catches it?	Advanced Expectation?
Numeric anomaly	Negative quantity in an order	✗ No	☑ Yes — qty > 0
Temporal inconsistency	Event date = year 1970 (system default)	✗ No	☑ Yes — date > '2000-01-01'
Range violation	Discount rate = 120%	✗ No	☑ Yes — discount BETWEEN 0 AND 100
Optional field rule	Field may be NULL, but when present must be >= 0	✗ No	☑ Yes — NULL-tolerant constraint
Schema evolution	New column added mid-stream breaks existing rules	✗ No	☑ Yes — CASE WHEN pattern
Data loss	Invalid records permanently dropped — no audit trail	✗ No	☑ Yes — Quarantine pattern

Real-World Example A discount_rate field containing 120 passes every NOT NULL check without complaint — but it represents an impossible discount that will silently corrupt downstream revenue calculations. Only a range constraint (discount_rate BETWEEN 0 AND 100) catches this before it flows downstream.

A2. Advanced Expectation Patterns — Cross-Table Validation

Beyond single-row constraints, expectations can validate data ACROSS tables using Materialized Views:

Row Count Validation

```
CREATE OR REFRESH MATERIALIZED VIEW count_verification (
  CONSTRAINT no_rows_dropped EXPECT (a_count == b_count)
```

```
    ON VIOLATION FAIL UPDATE
) AS SELECT * FROM
  (SELECT COUNT(*) AS a_count FROM table_a),
  (SELECT COUNT(*) AS b_count FROM table_b)
```

Missing Record Detection (LEFT OUTER JOIN)

```
CREATE OR REFRESH MATERIALIZED VIEW report_compare_tests (
  CONSTRAINT no_missing_records EXPECT (r_key IS NOT NULL)
    ON VIOLATION FAIL UPDATE
) AS SELECT v.*, r.key AS r_key
FROM validation_copy v
LEFT OUTER JOIN report r ON v.key = r.key
-- If r_key IS NULL → record exists in validation but MISSING in report
```

Primary Key Uniqueness Check

```
CREATE OR REFRESH MATERIALIZED VIEW report_pk_tests (
  CONSTRAINT unique_pk EXPECT (num_entries = 1)
    ON VIOLATION FAIL UPDATE
) AS SELECT pk, COUNT(*) AS num_entries
FROM report
GROUP BY pk
-- Any pk with COUNT > 1 = duplicate → fails constraint
```

A3. Writing NULL-Tolerant Constraints

When a column **may be absent** (optional field, or added via schema evolution), a basic range check will treat every NULL as a violation, causing widespread **false alarms**.

❌ **Naive (NULLs as violations)**

```
CONSTRAINT valid_discount
EXPECT (
  discount_rate >= 0
  AND discount_rate <= 100
)
-- Every NULL is flagged ❌
```

☑ **NULL-Tolerant (only validates when present)**

```
CONSTRAINT valid_discount
EXPECT (
  CASE
    WHEN discount_rate IS NOT NULL
    THEN discount_rate >= 0
         AND discount_rate <= 100
    ELSE TRUE  -- NULL is acceptable
  END
)
```

📌 **Rule of Thumb**

Whenever a column may be absent — because it is OPTIONAL or was added AFTER the pipeline started (schema evolution) — wrap its constraint in: CASE WHEN ... IS NOT NULL THEN ... ELSE TRUE END

B. Resilient Pipeline Design — Bronze Layer

B1. Ingest Everything as STRING

The most resilient Bronze layer **never rejects a record due to type mismatch**. Store all incoming fields as STRING at bronze, then enforce types safely at Silver using **TRY_CAST** (which returns NULL instead of failing on a bad cast).

B2. Two Schema Evolution Tools at Bronze Layer

Tool	What It Does	When to Use
schemaHints	Pre-declares columns you KNOW are coming in future files. Old records get NULL for those columns. Backward and forward compatible.	When you know which new columns are being added upstream.
_rescued_data	Catches ANY fields arriving outside the declared schema as JSON. Acts as a catch-all — nothing is silently lost.	When unexpected/unknown columns might arrive. Use for investigation and recovery.
schemaEvolutionMode = 'rescue'	Activates _rescued_data column in read_files(). Ensures no data is ever silently discarded.	Set at the Bronze read_files() call.

⚠ **Critical Interaction** When a column is added via schema evolution, ALL records ingested BEFORE the evolution carry NULL for that column. Any constraint written for that column MUST use the NULL-tolerant CASE WHEN pattern — otherwise every historic record fails the constraint, causing widespread false violations.

C. The Quarantine Pattern — Zero Data Loss

The **DROP ROW** mode permanently deletes invalid records — there is NO recovery path. The **Quarantine Pattern** eliminates data loss entirely by routing EVERY record into the table, then using an **is_quarantined** flag to split clean and failed records into separate downstream views.

🔑 **Core Guarantee** Total Records In = Clean Records + Quarantine Records NO record is EVER dropped. Every invalid record is preserved for investigation and reprocessing.

C1. The Quarantine Pattern — 2-Step Implementation

Step 1 — Quarantine Table with Inverse Logic (use WARN):

```
CREATE OR REFRESH STREAMING TABLE trips_quarantine (
  CONSTRAINT valid_distance EXPECT (trip_distance > 0),     -- WARN
  CONSTRAINT valid_fare     EXPECT (fare_amount >= 0),      -- WARN
  CONSTRAINT valid_pax      EXPECT (passenger_count BETWEEN 1 AND 9)   -- WARN
)
PARTITIONED BY (is_quarantined)
AS SELECT *,
  NOT(                              -- Inverse logic: TRUE if ANY rule fails
    trip_distance > 0
    AND fare_amount >= 0
    AND passenger_count BETWEEN 1 AND 9
  ) AS is_quarantined
FROM STREAM bronze_trips
```

Step 2 — Split into Clean and Failed Materialized Views:

```
-- Clean records for downstream analytics
CREATE OR REFRESH MATERIALIZED VIEW valid_trips_data
AS SELECT * FROM trips_quarantine WHERE is_quarantined = FALSE;

-- Failed records preserved for remediation and audit
CREATE OR REFRESH MATERIALIZED VIEW invalid_trips_data
AS SELECT * FROM trips_quarantine WHERE is_quarantined = TRUE;
```

⚠ Why WARN on the Quarantine Table? If DROP ROW or FAIL UPDATE were applied to the quarantine table, invalid records would be REMOVED before the is_quarantined flag is evaluated — defeating the entire pattern. WARN ensures EVERY record is written and routed correctly by the inverse logic column. WARN here is ONLY for surfacing per-constraint metrics in the Pipeline UI.

📌 Why Partition by is_quarantined? Partitioning physically separates clean and quarantined records on storage. Downstream queries filtering on is_quarantined = FALSE or TRUE then benefit from PARTITION PRUNING — only the relevant partition is scanned, not the entire table. This significantly improves read performance.

C2. DROP ROW vs Quarantine Pattern — When to Use Which

Feature	DROP ROW	Quarantine Pattern
Invalid records	Permanently deleted ❌	Preserved in quarantine table ☑
Audit trail	None ❌	Full — queryable ☑
Data recovery	Not possible ❌	Fix rule → re-route from quarantine ☑
UI violation metrics	Visible in pipeline UI	Per-constraint metrics via WARN ☑
Read performance	Full table scan on clean data	Partition pruning on is_quarantined ☑
Pipeline complexity	Low — single table	Moderate — 1 table + 2 views
Best for	Non-critical streams with well-established rules	Production pipelines with compliance, audit, or remediation needs

🏆 Recommendation For ENTERPRISE PRODUCTION pipelines, the Quarantine Pattern is the preferred approach. It gives you: • Zero data loss • Per-constraint violation metrics in the pipeline UI • Partition-pruned reads on clean data • Recoverable quarantine table for root cause analysis and reprocessing Capabilities that DROP ROW simply cannot provide.

EXAM QUICK REFERENCE — Chapter 5

High-Priority Exam Points

- **1.** Flow = Query + Target. Every flow independently reads, transforms, and writes data.
- **2.** RENAMING a flow RESETS its checkpoint — flow reprocesses ALL data from the beginning.
- **3.** Default Flow shares the same name as the target table. Created automatically.
- **4.** Explicit Flow (CREATE FLOW) is required for multi-flow patterns (multiple sources → one target).
- **5.** Multi-flow is BETTER than UNION because each flow has its OWN independent checkpoint. New sources = no full refresh.
- **6.** Data Quality Expectations CANNOT be defined on a Flow. Always defined on the TARGET TABLE.
- **7.** WARN = keep record (default). DROP ROW = delete record. FAIL UPDATE = halt entire pipeline.
- **8.** Liquid Clustering replaces Hive partitioning and Z-Ordering. Incremental, Flexible, Self-Tuning.
- **9.** CLUSTER BY AUTO = Databricks chooses keys. Starts unset — needs query observation first.
- **10.** Multiplex Pattern = Ingest all event types into ONE bronze table, then fan out by type downstream.
- **11.** ONLY Python API supports Sinks. SQL is NOT supported. Only append_flow writes to a Sink.
- **12.** Streaming Tables and Materialized Views CANNOT have Iceberg UniForm. Use a Delta Sink (plain Delta table) instead.
- **13.** 4 Delta table properties needed for UniForm: enableDeletionVectors=false, columnMapping.mode=name, enableIcebergCompatV2=true, universalFormat.enabledFormats=iceberg.
- **14.** AUTO CDC INTO = renamed APPLY CHANGES INTO. Syntax is IDENTICAL.
- **15.** SCD Type 1 = overwrite (no history). SCD Type 2 = preserve history (__START_AT, __END_AT columns).
- **16.** __END_AT = NULL means CURRENTLY ACTIVE record in SCD Type 2.
- **17.** SCD Type 2 DELETE = SOFT DELETE — row stays but __END_AT is populated.
- **18.** NULL-Tolerant Constraint: CASE WHEN column IS NOT NULL THEN <rule> ELSE TRUE END.
- **19.** Quarantine Pattern uses WARN (not DROP ROW) on the quarantine table — so inverse logic can route ALL records.
- **20.** is_quarantined = NOT(all rules pass). Partition by is_quarantined for performance.
- **21.** TRY_CAST returns NULL instead of failing — use at Silver layer for safe type enforcement.
- **22.** _rescued_data captures unexpected schema fields as JSON — nothing is ever silently discarded.

Master Comparison Table — All Patterns

Pattern / Concept	What Problem It Solves	Key Rule to Remember

Default Flow	Simple 1-source to 1-target pipelines	Flow name = Table name. Auto-created with table.
Explicit Flow (CREATE FLOW)	Multiple sources writing to one target	Required for multi-flow. Defined separately from table.
Multi-Flow Pattern	Adding new sources without full refresh	Each flow = own checkpoint. Better than UNION.
Liquid Clustering	Slow query performance due to unoptimized data layout	Replaces Hive partitioning + Z-Ordering. Incremental optimization.
CLUSTER BY AUTO	Unknown query patterns on new tables	Databricks auto-selects keys. Starts unset.
WARN / DROP ROW / FAIL UPDATE	Data quality enforcement	WARN = keep. DROP = delete. FAIL = halt pipeline.
Multiplex Pattern	Multiple event types in one source stream	Ingest once into bronze → fan out by type downstream.
Delta Sink	Writing to external targets / enabling Iceberg	Python only. Only append_flow. External Delta table.
Delta UniForm	Cross-platform Iceberg access on Delta tables	4 properties. No data duplication. Read-only for Iceberg clients.
SCD Type 1	Only current state needed	OVERWRITE. No history. STORED AS SCD TYPE 1.
SCD Type 2	Full history + audit trail needed	NEW row per change. __END_AT = NULL for active rows.
AUTO CDC INTO	Declarative CDC without complex MERGE logic	Same as APPLY CHANGES INTO. Handles ordering via SEQUENCE BY.
NULL-Tolerant Constraint	Avoiding false violations from schema evolution or optional fields	CASE WHEN col IS NOT NULL THEN <rule> ELSE TRUE END
Quarantine Pattern	Zero data loss with full audit trail for invalid records	WARN on quarantine table. Inverse logic. Partition by is_quarantined.

📖 COMPLETE GLOSSARY — Chapter 5 Terms

- **append_flow** — The ONLY flow type that can write to a Sink. Defined with @dp.append_flow in Python. No SQL equivalent.
- **AUTO CDC INTO** — Declarative CDC syntax in Spark Declarative Pipelines (formerly APPLY CHANGES INTO).
- **Bronze Layer** — Raw ingestion layer in Medallion architecture. Best practice: store all fields as STRING.
- **CLUSTER BY AUTO** — Databricks auto-selects optimal clustering keys based on observed query patterns.
- **CLUSTER BY (columns)** — Explicit clustering keys you specify. Best when query patterns are stable and known.
- **Checkpoint** — A stored pointer tracking how far a flow has read from its source. Enables reliable restarts.
- **Composite Key** — Using multiple columns together as a unique identifier. E.g., KEYS (CustomerID, OrderID).
- **CONSTRAINT ... EXPECT** — Syntax for defining a data quality rule on a Streaming Table in SDP.
- **Cross-Table Validation** — Expectations on Materialized Views that validate data across multiple tables.
- **Default Flow** — Auto-created flow that shares its name with the target table.
- **Delta Sink** — A Sink that writes to an external Delta table outside the pipeline's managed scope.
- **Delta UniForm** — Auto-generates Iceberg metadata on Delta tables' Parquet files for cross-platform reads.
- **Deletion Vectors** — Delta soft-delete feature. MUST be disabled for UniForm/Iceberg compatibility.
- **DROP ROW** — Violation mode: invalid rows are permanently removed from the target table.
- **Explicit Flow** — A flow defined separately using CREATE FLOW. Required for multi-flow patterns.
- **FAIL UPDATE** — Violation mode: entire pipeline update is halted on first rule violation.
- **Fan-Out** — Splitting one ingested bronze table into multiple domain-specific tables by event type.
- **Flow** — Basic unit of work in SDP. Reads, transforms, and writes data to a target.
- **Full Refresh Mode** — Discards checkpoint and reprocesses ALL records from scratch. Overwrites target.
- **Incremental Mode** — Processes only NEW records since the last run using a stored checkpoint.
- **Inverse Logic** — is_quarantined = NOT(all rules). TRUE if any rule fails — used in Quarantine Pattern.
- **is_quarantined** — Boolean flag column in the Quarantine Pattern. TRUE = failed at least one quality rule.
- **Liquid Clustering** — Delta Lake optimization replacing Hive partitioning and Z-Ordering. Incremental, flexible, self-tuning.
- **Missing Record Detection** — LEFT OUTER JOIN-based validation to find records absent in the target but present in a reference copy.
- **Multiplex Pattern** — Ingesting all event types into ONE bronze table, then fanning out by type downstream.
- **NULL-Tolerant Constraint** — CASE WHEN col IS NOT NULL THEN <rule> ELSE TRUE END — avoids false violations for absent fields.
- **ON VIOLATION** — Clause appended to EXPECT to specify WARN, DROP ROW, or FAIL UPDATE.

- **Partition Pruning** — Query optimization scanning only the relevant partition. Used in Quarantine Pattern (partitioned by is_quarantined).
- **Primary Key Uniqueness** — GROUP BY pk + EXPECT (COUNT = 1) check to detect duplicate keys.
- **Quarantine Pattern** — Routes ALL records to a table with an is_quarantined flag, then splits into clean/invalid views. Zero data loss.
- **Row Count Validation** — Comparing COUNT(*) between two tables to detect silent record loss.
- **SCD Type 1** — Overwrite mode — only current state stored. No history. STORED AS SCD TYPE 1.
- **SCD Type 2** — History-preserving mode — new row per change. __START_AT and __END_AT columns track validity periods.
- **SEQUENCE BY** — Column in AUTO CDC INTO that orders CDC events chronologically. Handles out-of-order events.
- **Sink** — Mechanism to write pipeline data to targets OUTSIDE the pipeline's managed scope.
- **schemaHints** — Pre-declares upcoming columns in read_files(). Old records get NULL for declared-but-absent columns.
- **schemaEvolutionMode = 'rescue'** — Activates the _rescued_data catch-all column in read_files().
- **Spark Declarative Pipelines (SDP)** — Databricks framework for building reliable, scalable data pipelines using declarative SQL or Python.
- **Streaming Table** — SDP dataset type for incremental, append-only data. Supports expectations and CDC.
- **TRY_CAST** — Safe type casting that returns NULL instead of raising an error on failure. Used at Silver layer.
- **WARN (default)** — Violation mode: invalid rows KEPT in table. Warning metric logged. No ON VIOLATION clause needed.
- **_rescued_data** — Catch-all column capturing unexpected/schema-mismatched fields as JSON. Ensures zero data discards.
- **__END_AT** — SCD Type 2 column. NULL = currently active. Non-null = historical/inactive record.
- **__START_AT** — SCD Type 2 column. Timestamp when this version of the row became active.

— End of Chapter 5 Study Guide —